Objective

Train a *Random Forest* on a *real-world*-dataset on *heart failure prediction*.

*# Submission Guidelines Your finished *Jupyter Notebook* - both as `.ipynb` and exported `.pdf`.

Background

From the *World Health Organization*:

- Cardiovascular diseases (CVDs) are the leading cause of death globally.
- An estimated 17.9 million people died from CVDs in 2019, representing 32% of all global deaths. Of these deaths, 85% were due to heart attack and stroke.
- Over three quarters of CVD deaths take place in low- and middle-income countries.
- Out of the 17 million premature deaths (under the age of 70) due to noncommunicable diseases in 2019, 38% were caused by CVDs.
- Most cardiovascular diseases can be prevented by addressing behavioural risk factors such as tobacco use, unhealthy diet and obesity, physical inactivity and harmful use of alcohol.
- It is important to detect cardiovascular disease as early as possible so that management with counselling and medicines can begin.

Apart from `Age` and `Sex`, you are given the following attributes to predict whether a person has suffered (or is likely to suffer) from a `HeartDisease` (1) or not (0):

- `ChestPainType`: type of chest pain.
  - *TA*: Typical Angina
  - *ATA*: Atypical Angina
  - *NAP*: Non-Anginal Pain
  - *ASY*: Asymptomatic
- `RestingBP`: resting blood pressure in *mmHg*.
- `Cholesterol`: serum cholesterol in *mm/dl*.
- `FastingBS`: 1 if the the fasting blood sugar is above 120 *mg/dl*.
- `RestingECG`: resting electrocardiogram results.
  - *Normal*: Normal
  - *ST*: having *ST-T* wave abnormality (*T wave inversions* and/or *ST elevation* or depression higher than 0.05 *mV*)
  - *LVH*: showing probable or definite *left ventricular hypertrophy* by *Estes' criteria*

- MaxHR : maximum heart rate achieved.
- ExerciseAngina : *Yes*, if the person suffers from exercise-induced angina.
- Oldpeak : measure of the depression occurring in the *ST* segment (*mm*).
- ST_Slope : the slope of the peak exercise *ST* segment.
  - *Up*: upsloping
  - *Flat*: flat
  - *Down*: downsloping

Task

1. Load the data from the provided `.csv` -**files** into `X_train`, `y_train` and `X_test`.
   - This time you don't have access to the *test labels*!
2. Perform a quick *EDA* (*Exploratory Data Analysis*).
3. Create 2-3 visualizations to illustrate aspects of the data you think are interesting!
4. Encode the *categorical variables* as you deem suitable.
   - As always, only `fit` the *encoder* on the training data, not the test data!
   - Hint: You'll need both *ordinal* and *one-hot* encoding!
5. Feel free to apply any further preprocessing steps.
6. Train and evaluate a *random forest* using *cross validation* on `X_train`.
7. Train a model using the best combination of hyperparameters and preprocessing steps on all of `X_train` and make predictions on `X_test`.
8. You'll get `y_test` in the next lesson, to see how well your model performs on unseen data!

Use the *random state* 12 whereever possible so we can compare our results across the class!

In [1]:
```
my_random_state = 12
```

1. Loading The Data

In [2]:
```
import pandas as pd
X_train = pd.read_csv('X_train.csv', index_col=0)
y_train = pd.read_csv('y_train.csv', index_col=0)['HeartDisease']
X_test = pd.read_csv('X_test.csv', index_col=0)
```

2. EDA

In [3]:
```
# Schnelle Überprüfung der Struktur und fehlenden Werte
print(X_train.info())
print(X_train.describe())
print(y_train.value_counts(normalize=True))
```

```
<class 'pandas.DataFrame'>
Index: 688 entries, 282 to 76
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Age             688 non-null    int64  
 1   Sex             688 non-null    str    
 2   ChestPainType   688 non-null    str    
 3   RestingBP       688 non-null    int64  
 4   Cholesterol     688 non-null    int64  
 5   FastingBS       688 non-null    int64  
 6   RestingECG      688 non-null    str    
 7   MaxHR           688 non-null    int64  
 8   ExerciseAngina  688 non-null    str    
 9   Oldpeak         688 non-null    float64
 10  ST_Slope        688 non-null    str    
dtypes: float64(1), int64(5), str(5)
memory usage: 64.5 KB
None
              Age   RestingBP  Cholesterol   FastingBS       MaxHR     Old
peak
count  688.000000  688.000000   688.000000  688.000000  688.000000  688.00
0000
mean    53.546512  132.306686   197.202035    0.229651  136.366279    0.88
2267
std      9.426226   18.641266   110.338086    0.420915   25.858975    1.07
2553
min     29.000000    0.000000     0.000000    0.000000   60.000000   -2.60
0000
25%     47.000000  120.000000   169.000000    0.000000  119.000000    0.00
0000
50%     54.000000  130.000000   223.000000    0.000000  137.500000    0.60
0000
75%     60.000000  140.000000   267.250000    0.000000  156.000000    1.50
0000
max     77.000000  200.000000   603.000000    1.000000  202.000000    6.20
0000
HeartDisease
1    0.553779
0    0.446221
Name: proportion, dtype: float64
```

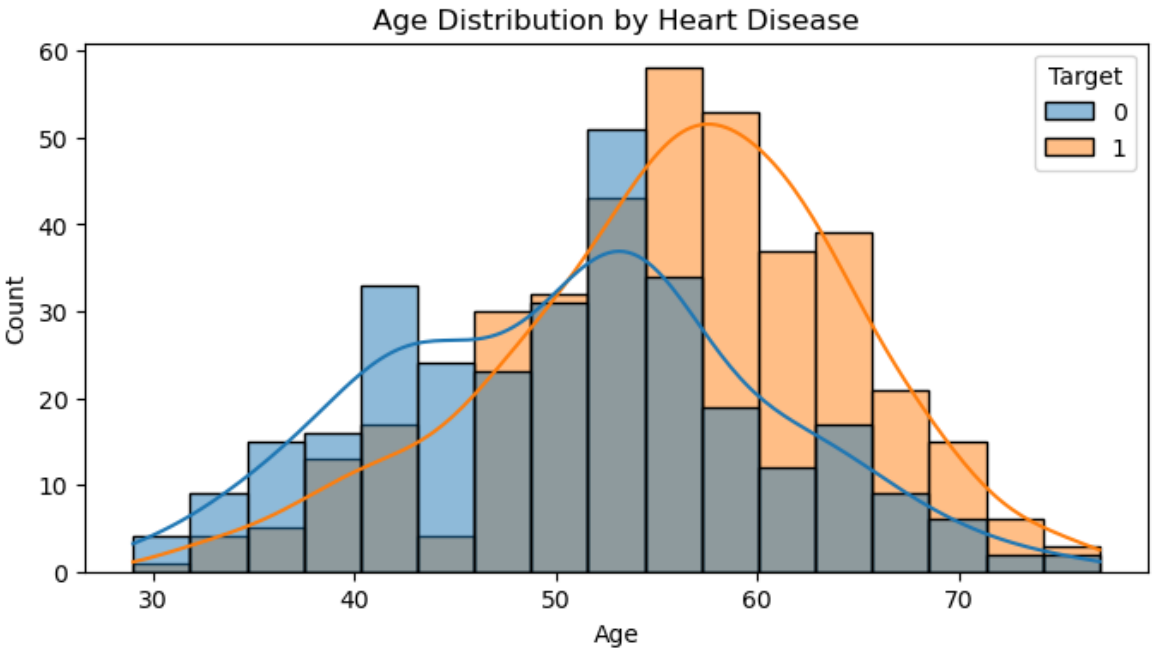
3. Vizualization

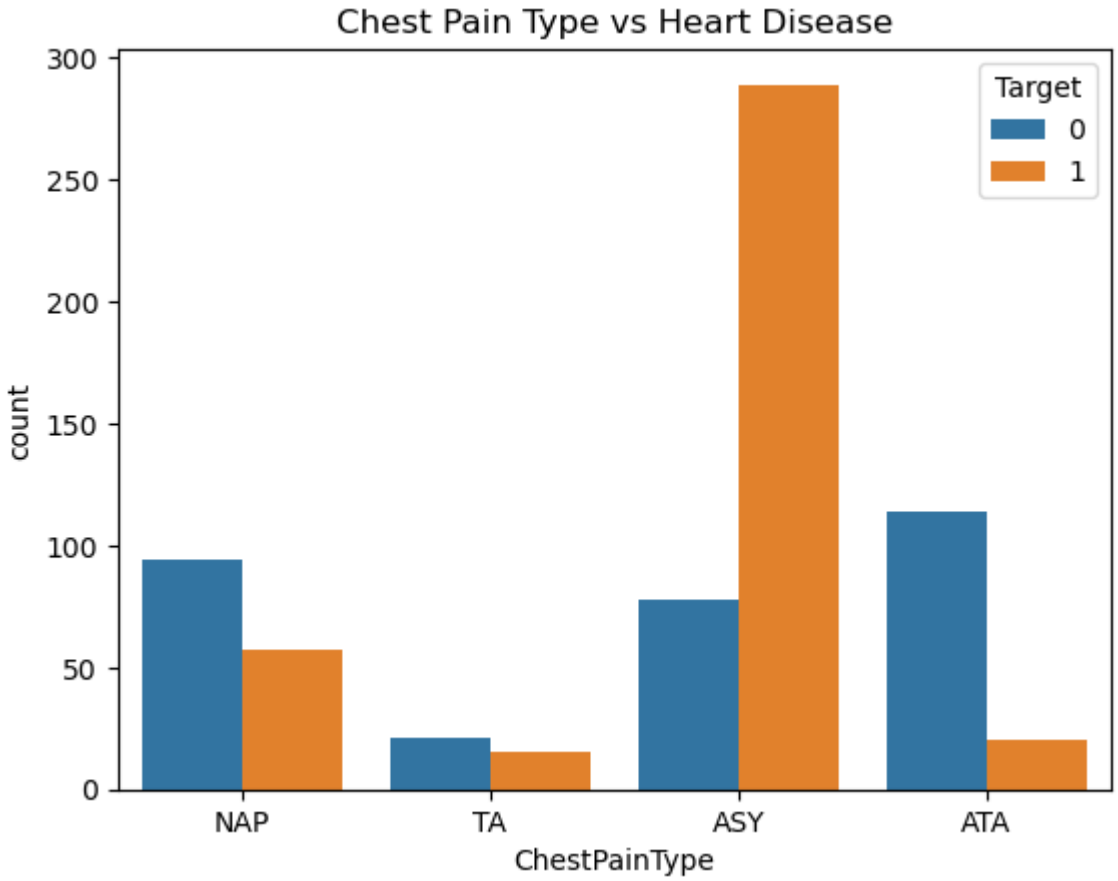
In [4]:
```
%pip install seaborn
import matplotlib.pyplot as plt
import seaborn as sns

# Verteilung des Alters vs. Herzerkrankung
plt.figure(figsize=(8, 4))
sns.histplot(data=X_train.assign(Target=y_train), x='Age', hue='Target',
plt.title('Age Distribution by Heart Disease')
plt.show()

# Brustschmerztyp vs. Herzerkrankung
sns.countplot(data=X_train.assign(Target=y_train), x='ChestPainType', hue
plt.title('Chest Pain Type vs Heart Disease')
plt.show()
```

```
Defaulting to user installation because normal site-packages is not writea
ble
Requirement already satisfied: seaborn in /home/spizza/.local/lib/python
3.13/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/miniconda3/li
b/python3.13/site-packages (from seaborn) (2.4.4)
Requirement already satisfied: pandas>=1.2 in /opt/miniconda3/lib/python
3.13/site-packages (from seaborn) (3.0.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/miniconda3/
lib/python3.13/site-packages (from seaborn) (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in /opt/miniconda3/lib/pyt
hon3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/miniconda3/lib/python
3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/miniconda3/lib/py
thon3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/miniconda3/lib/py
thon3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /opt/miniconda3/lib/pyth
on3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
Requirement already satisfied: pillow>=8 in /opt/miniconda3/lib/python3.1
3/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.1.1)
Requirement already satisfied: pyparsing>=3 in /opt/miniconda3/lib/python
3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /opt/miniconda3/li
b/python3.13/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/lib/python3.13/
site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seabor
n) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```





4. Encoding

In [5]:
```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer

# Spalten definieren
cat_ohe = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina']
cat_ord = ['ST_Slope'] # Aufwärtshang/Flach/Abwärtshang hat eine inhärent
num_cols = ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Old

# Transformer einrichten
preprocessor = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    ('ord', OrdinalEncoder(categories=[['Down', 'Flat', 'Up']]), cat_ord)
], remainder='passthrough')

# Anpassen und transformieren
X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc = preprocessor.transform(X_test)
```

5. Furhter data preprocessing

In [6]:
```
# Optional: Skalierung (nicht unbedingt erforderlich)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_proc = scaler.fit_transform(X_train_proc)
X_test_proc = scaler.transform(X_test_proc)
```

6. Train and evaluate RF using CV on X_train

In [7]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rf_cv = RandomForestClassifier(random_state=12)
cv_scores = cross_val_score(rf_cv, X_train_proc, y_train, cv=5)
print(f"CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")
```

```
CV Accuracy: 0.8518 (+/- 0.0173)
```

7. Train a model using the best combination of hyperparameters and preprocessing

In [8]:
```
from sklearn.model_selection import GridSearchCV

# Einfache Hyperparameter-Abstimmung
params = {'n_estimators': [100, 200], 'max_depth': [None, 10, 20]}
grid = GridSearchCV(RandomForestClassifier(random_state=12), params, cv=5
grid.fit(X_train_proc, y_train)

# Vorhersagen auf X_test
y_pred = grid.best_estimator_.predict(X_test_proc)
print("Best Params:", grid.best_params_)
```

```
Best Params: {'max_depth': 10, 'n_estimators': 200}
```

8. Check how well your model performs on unseen data finally using y_test

In [9]:
```
from sklearn.metrics import classification_report, confusion_matrix, accu

y_test = pd.read_csv('y_test.csv', index_col=0)['HeartDisease']

# Vergleichen Sie mit den Vorhersagen (y_pred) aus Schritt 7
accuracy = accuracy_score(y_test, y_pred)
print(f"Final Test Accuracy: {accuracy:.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Final Test Accuracy: 0.9043

Confusion Matrix:
[[ 92  11]
 [ 11 116]]

Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.89      0.89       103
           1       0.91      0.91      0.91       127

    accuracy                           0.90       230
   macro avg       0.90      0.90      0.90       230
weighted avg       0.90      0.90      0.90       230
```